

요구사항 #3: 상태 기반 UI 표현 + CI/CD (GitHub Actions + Render)

주의: 이 요구사항은 요구사항 #2 진행 중에 전달됩니다. 실제 업무에서도 기능 개발 도중 새로운 요구사항이 들어오는 경우가 빈번합니다. 브랜치 전략을 활용하여 두 작업을 관리하세요.

목표

상태 값에 따라 색상(badge)을 구분하여 표시하고,
GitHub Actions 도 CI(자동 테스트) 를 자동화한 뒤,
Render 도 CD(자동 배포) 까지 연결하여 전체 파이프라인을 완성합니다.

역할별 작업

역할	작업 내용
PM	요구사항을 FE 에 배분, GitHub Actions CI 설정, 최종 배포
FE	<code>badgeClass()</code> 함수 구현 (상태별 색상 매핑)
BE	(이번 요구사항에서는 추가 구현 없음 — PM 의 CI 설정 지원)
TE	상태 색상 검증 시나리오 추가 → 검증 → 최종 Report 작성

완료 조건

상태 Badge (4.7)

- 모든 상태 값에 적절한 색상 badge 가 적용된다
- Active/Online/Normal → 초록
- Paused/Standby → 파랑
- Expired/Error/Warning → 빨강
- Offline → 회색

GitHub Actions CI (4.8)

- `.github/workflows/ci.yml` 워크플로우가 설정된다
- Push/PR 시 자동으로도 CI 가 실행된다
- CI 에서 서버 기동 + API 테스트가 동작한다

Render CD (자동 배포)

- Render Web Service 가 본인의 GitHub 저장소와 연결된다
- main 브랜치에 push/merge 시 자동으로도 재배포(Auto-Deploy)된다
- 배포 URL(<https://<서비스이름>.onrender.com>)에서 전체 기능이 동작한다
- (선택) CI 통과 시에만 배포되도록 Render Deploy Hook 을 연동한다

검증

- TE 의 최종 검증 Report 가 작성되었다 (요구사항 #1~#3 전체)

Part A: 상태 기반 UI 표현 (FE)

구현 가이드

파일: `app/static/app.js`

`badgeClass(value)` 함수를 구현하세요.

```
function badgeClass(value) {  
  const v = (value || "").toLowerCase();  
  
  // 매핑 규칙:  
  // Active, Online, Normal → "badge status-active" (초록)  
  // Paused, Standby → "badge status-paused" (파랑)  
  // Expired, Error, Warning → "badge status-expired" (빨강)
```

```
// Offline      → "badge status-offline" (회색)
// On, Cleaning  → "badge status-on" (노당)
// Off          → "badge status-off" (연회색)
// 그 외        → "badge"
}
```

색상 매핑 표

상태 값	색상	CSS 클래스	적용 위치
Active, Online, Normal	초록	status-active	구독 상태, 가전 상태, 건강 상태
Paused, Standby	파랑	status-paused	구독 상태, 가전 상태
Expired, Error, Warning	빨강	status-expired	구독 상태, 가전 상태, 건강 상태
Offline	회색	status-offline	가전 상태
On, Cleaning	노당	status-on	전원 상태
Off	연회색	status-off	전원 상태

Part B: GitHub Actions CI 설정 (PM)

Step 1: 워크플로우 파일 생성

파일: [.github/workflows/ci.yml](#)

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  ci:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run syntax check
        run: |
          python -m py_compile app/main.py
          python -m py_compile app/api/subscribers.py
          python -m py_compile app/api/devices.py
```

```

- name: Start server and test health endpoint
  run: |
    uvicorn app.main:app --host 0.0.0.0 --port 8000 &
    sleep 3
    curl -f http://localhost:8000/health

- name: Test API endpoints
  run: |
    curl -f http://localhost:8000/api/subscribers
    curl -f http://localhost:8000/api/subscribers/U001/devices
    curl -f http://localhost:8000/api/devices/D001/usage

```

Step 2: Render Deploy Hook 설정 (선택)

Render Auto-Deploy 를 끄고, CI 통과 시에만 배포하려면:

```

- name: Trigger Render Deploy
  if: success() && github.ref == 'refs/heads/main'
  run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK_URL }}"

```

Render Dashboard → Service → Settings → Deploy Hook 에서 URL 을 복사하여

GitHub Repository → Settings → Secrets 에 RENDER_DEPLOY_HOOK_URL 도 등록하세요.

Step 3: 커밋 및 Push

```

# 1. 워크플로우 파일 생성
mkdir -p .github/workflows

# 2. ci.yml 파일 작성 (위 내용 참고)

# 3. 커밋
git add .github/workflows/ci.yml app/static/app.js
git commit -m "feat: add status badges and GitHub Actions CI"

# 4. Push
git push origin main

```

Step 4: GitHub 에서 CI 확인

1. GitHub 리포지토리 → **Actions** 탭 이동
2. 워크플로우 실행 토글 확인
3. 모든 Step 이 초록색(통과)인지 확인

Part C: Render 배포 (CD)

CI 가 "코드가 안전한지 자동으로 검사"하는 것이라면,

CD 는 "검사를 통과한 코드를 자동으로 서비스에 배포"하는 것입니다.

Render 는 GitHub 저장소와 연결해두면 **main** 에 **push/merge** 될 때마다 자동으로 재배포합니다.

Step 1: Render 가입 및 로그인

1. <https://render.com/> 접속 → **Start for free** → 계정 가입 → 로그인
- GitHub 계정으로 가입하면 이후 연동이 편리합니다.

Step 2: Web Service 생성

1. 우측 상단 **New** → **Web Services** 선택
-  **Static Site** 가 아닌 **Web Services** 를 선택하세요.
(FastAPI 는 동적 웹앱/API 서버이므로 Web Services)

Create a new Static Site

1 Choose service > 2 Configure > 3 Deploy

Static Sites

Static content served over a global CDN. Ideal for frontend, blogs, and content sites.

[New Static Site →](#)

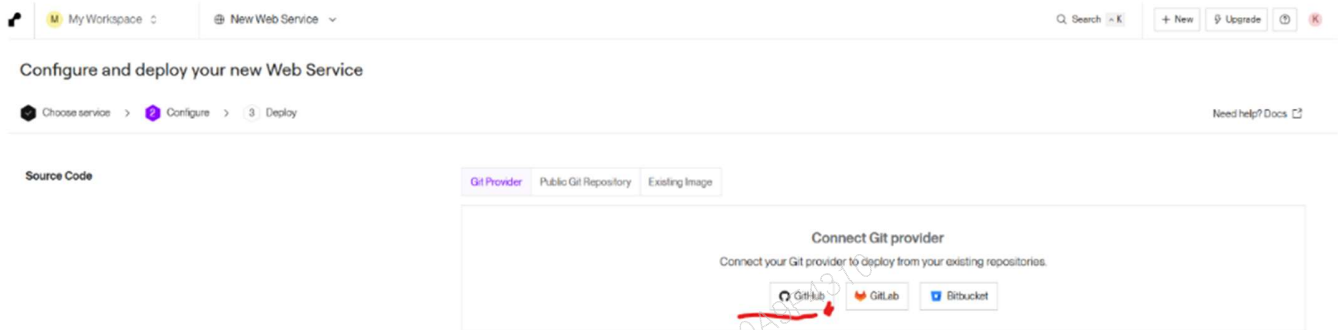
Web Services

Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.

[New Web Service →](#)

Step 3: GitHub 연결

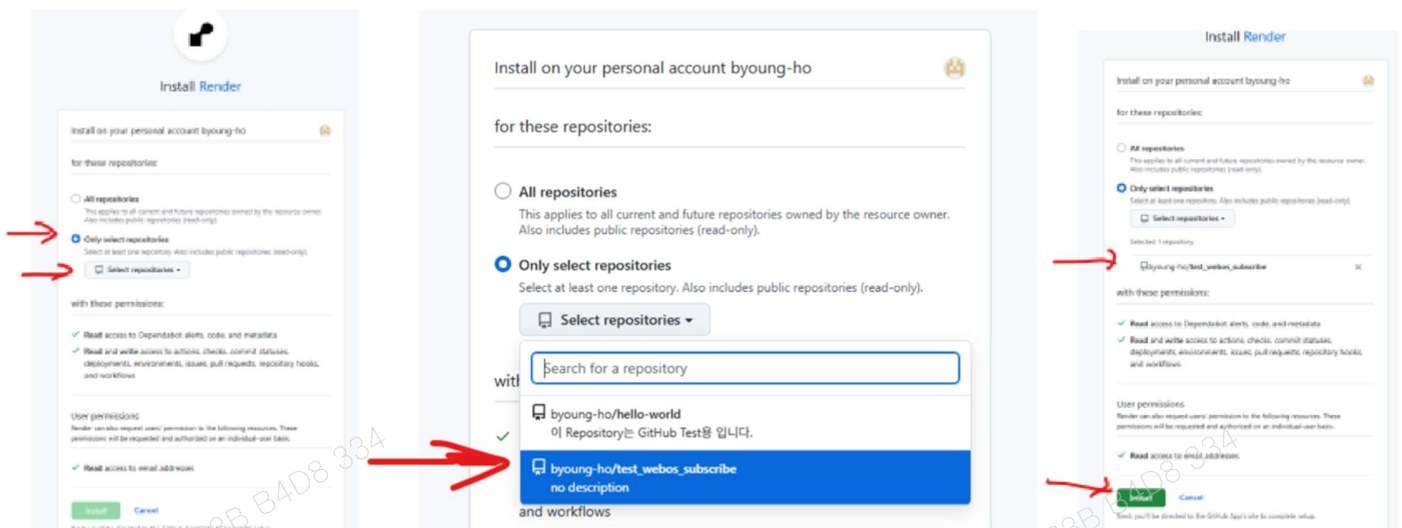
1. Source Code → Git Provider → GitHub 선택
2. Connect Git provider 도 본인 GitHub 계정은 연결



Step 4: Render 앱 설치 (⚠ 매우 중요)

1. Authorized → Install Render 진행
2. Only select repositories 선택
3. Select repositories 에서 본인이 작업 중인 저장소(예: test_webos_subscribe)를 선택
4. 확인 후 Install

💡 All repositories 대신 Only select repositories 도
필요한 저장소만 연결하는 것이 안전합니다.



Step 5: 배포 설정

Render 의 Configure and deploy your new Web Service 화면에서

연결한 저장소를 선택한 뒤, 아래 2 가지만 확인/수정하고 나머지는 기본값 그대로 둡니다.

설정 항목	값
Name	test_webos_subscribe (원하는 이름)
Language	Python 3
Branch	main
Region	Oregon (US West) 등 기본값
Build Command	pip install -r requirements.txt
Start Command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Instance Type	Free

⚠ **Start Command 주의:** 포트는 반드시 \$PORT도 지정해야 합니다. 8000 같은 특정 포트를 직접 넣으면 배포에 실패합니다. (Render가 런타임에 포트를 주입하므로 \$PORT를 사용)

Source Code
tryoung-ho / test_webos_subscribe • 12m ago
Edit

Name
test_webos_subscribe

Language
Python 3

Branch
main

Region
Oregon (US West)

Root Directory
Optional
0 / . / src

Build Command
\$ pip install -r requirements.txt

Start Command
\$ uvicorn app.main:app --host 0.0.0.0 --port \$PORT

💡 **Instance Type: Free** – 512 MB RAM / 0.1 CPU / \$0. 실습에는 충분합니다.
(무료 인스턴스는 미사용 시 sleep 되어 첫 접속이 느릴 수 있음)

Instance Type

For hobby projects
Free
512 MB (RAM)
0.1 CPU
\$0 / month
Upgrade to enable more features
Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

For professional use
Starter
512 MB (RAM)
0.5 CPU
\$7 / month
Standard
2 GB (RAM)
1 CPU
\$25 / month
Pro
4 GB (RAM)
2 CPU
\$85 / month
Pro Plus
8 GB (RAM)
4 CPU
\$175 / month
Pro Max
16 GB (RAM)
4 CPU
\$225 / month
Pro Ultra
32 GB (RAM)
8 CPU
\$450 / month
Need a custom instance type? We support up to 512 GB RAM and 64 CPUs.

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. Learn more.
NAME_OF_VARIABLE
value
Generate
Add Environment Variable
Add from env

Advanced
Deploy Web Service

Step 6: 배포 실행

1. 화면 하단 Deploy Web Service 버튼 클릭
2. 빌드 → 배포까지 약 3~5 분 소요

WEB SERVICE

test_webos_subscribe Python3 Free Upgrade your instance →

Service ID: srv-d9vsteuuk2gs738v3t9g

byoung-ho / test_webos_subscribe ~1 main

<https://test-webos-subscribe.onrender.com>

Connect Manual Deploy

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. Upgrade now

March 22, 2026 at 7:43 PM Live

#06b02 Init webos subscription service

All logs Search Mar 22 7:42 PM GMT+9

```
07:40:14 PM [teagph] --- Build successful 🎉
07:40:20 PM [teagph] --- Deploying...
07:40:20 PM [teagph] --- Setting WEB_CONCURRENCY:1 by default, based on available CPUs in the instance
07:40:25 PM [teagph] --- No open ports detected, continuing to scan...
07:40:35 PM [teagph] --- Done on specifying a port: https://render.com/docs/web-services#port-binding
07:40:40 PM [teagph] --- Running 'uvicorn app.main:app --host 0.0.0.0 --port $PORT'
07:40:46 PM [teagph] INFO: Started server process [57]
07:40:46 PM [teagph] INFO: Waiting for application startup.
07:40:46 PM [teagph] INFO: Application startup complete.
07:40:46 PM [teagph] INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
07:40:47 PM [teagph] INFO: 127.0.0.1:54068 - "HEAD / HTTP/1.1" 405 Method Not Allowed
07:40:51 PM [teagph] INFO: 116.45.59.178 - "GET /favicon.ico HTTP/1.1" 404 Not Found
07:40:51 PM [teagph] INFO: 116.45.59.178 - "HEAD / HTTP/1.1" 405 Method Not Allowed
07:40:51 PM [teagph] INFO: 116.45.59.178 - "GET / HTTP/1.1" 200 OK
07:40:52 PM [teagph] --- Your service is live 🎉
07:40:52 PM [teagph] ---
07:40:52 PM [teagph] --- Available at your primary URL https://test-webos-subscribe.onrender.com
07:40:52 PM [teagph] ---
```

Step 7: 배포 확인

1. 도그에 다음 메시지가 뜨면 성공입니다.

==> Build successful 🎉

==> Deploying...

==> Running 'uvicorn app.main:app --host 0.0.0.0 --port \$PORT'

INFO: Application startup complete.

==> Your service is live 🎉

==> Available at your primary URL <https://<서비스이름>.onrender.com>

2. 제공된 배포 URL 확인:

- <https://<서비스이름>.onrender.com> → 대시보드 화면
- <https://<서비스이름>.onrender.com/health> → {"status": "ok"}

test-webos-subscribe.onrender.com

webOS Subscription Management Dashboard

Yonsei University DevOps Practice - Dummy Subscription Management Service

Subscribers

User ID	Name	Plan	Status	Devices
U001	Kim Minsoo	Premium	Active	2
U002	Lee Jiyeon	Basic	Active	1
U003	Park Junho	Family	Paused	3
U004	Choi Sumin	Premium	Active	2
U005	Jung Hyerin	Basic	Expired	0

Devices

Device ID	Type	Model	Location	Status
D004	TV	LG QNED	Lab	Online
D005	Air Conditioner	LG Whisen	Office	Online
D006	Styler	LG Styler	Home	Error

Usage Detail

Device ID: D005

Device Name: LG Whisen

Power Status: On

Last Used: 2026-03-22 10:58:00

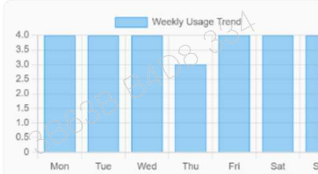
Total Usage Hours: 310

Weekly Usage Count: 27

Health Status: Normal

Remark: Temperature control active

Weekly Usage Trend



Step 8: CD(자동 배포) 동작 확인

- 1. 코드를 수정하고 main 에 push (또는 PR merge)
- 2. Render 가 자동으도 재배포하는지 Events/Logs 탭에서 확인
- 3. 배포 완료 후 URL 에서 변경사항이 반영되었는지 확인

✓ 여기까지 하면 push → (GitHub Actions CI) → Render 자동 배포(CD) 파이프라인이 완성됩니다.

TE: 테스트 시나리오 (예시)

상태 Badge (4.7)

#	테스트 시나리오	예상 결과	결과	비고
1	Active 상태 구독자 확인	초독 badge		
2	Paused 상태 구독자 확인	파당 badge		
3	Expired 상태 구독자 확인	빨강 badge		
4	Online 상태 가전 확인	초독 badge		
5	Offline 상태 가전 확인	회색 badge		
6	Error 상태 가전 확인	빨강 badge		
7	Power On 상태 확인	노랑 badge		
8	Health Normal 상태 확인	초독 badge		
9	Health Warning 상태 확인	빨강 badge		

CI 파이프라인 (4.8)

#	테스트 시나리오	예상 결과	결과	비고
10	main push 시 CI 자동 실행	Actions 탭에서 실행 확인		
11	CI 에서 health 체크 통과	초독 체크마크		
12	CI 에서 API 테스트 통과	3 개 엔드포인트 모두 통과		
13	CI 통과 후 Render 배포 확인	배포 URL 접속 가능		

DevOps 실습 포인트

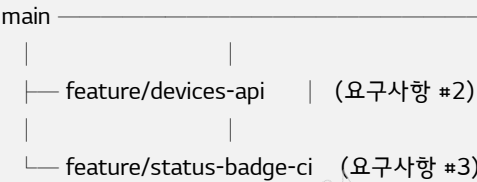
브랜치 관리 (요구사항 #2 진행 중이라면)

```
# 요구사항 #2 작업 중이라면, 현재 작업을 커밋하거나 stash
git stash
# 또는
git add . && git commit -m "wip: devices API in progress"

# main 브랜치로 이동 후 새 브랜치 생성
git checkout main
git checkout -b feature/status-badge-ci

# 구현 후 커밋 + Push + PR
```

브랜치 상태



두 브랜치가 같은 파일(app.js)을 수정하므로, merge 시 충돌(conflict)이 발생할 수 있습니다.
이는 의도된 학습 포인트입니다.

최종 파이프라인 확인

코드 수정



git push (또는 PR merge)



GitHub Actions 자동 실행 (CI)

├─ 코드 체크아웃

├─ Python 설정

├─ 의존성 설치

├─ 문법 검사

└─ API 테스트 (서버 기동 + curl)



CI 통과 ✔



Render 배포 (CD)



배포 URL 에서 전체 기능 확인

전체 실습 완료 체크리스트

- ☐ GET /api/subscribers 동작 확인 (요구사항 #1)
- ☐ GET /api/subscribers/{userId}/devices 동작 확인 (요구사항 #2)
- ☐ GET /api/devices/{deviceId}/usage 동작 확인 (요구사항 #2)
- ☐ 상태 badge 색상 적용 확인 (요구사항 #3)
- ☐ GitHub Actions CI 자동 실행 확인 (요구사항 #3)
- ☐ Render 배포 URL 에서 전체 기능 동작 확인
- ☐ 검증 Report 제출 완료